1. Load the Data in PySpark

```
df = spark.read.csv(
    "manufacturing_production_120k.csv",
    header=True,
    inferSchema=True
)
```

Initial exercises:

1. Display first 20 rows.
2. Display schema.
3. Display column names.
4. Count total records.
5. Display only production ID, plant, machine, product family and units produced.
6. Check the datatype of each column.
7. Display 10 random-looking sample records using `sample()` .

---

2. Creating DataFrames

Create a small DataFrame manually containing:

```
plant_id
plant_name
city
```

Then:

1. Create it using a list of tuples.
2. Create it with explicit column names.
3. Create another version using `StructType` .
4. Print the schema.
5. Add a constant column called `country = India` .

---

3. Pandas DataFrame

Read the same main CSV using Pandas:

```
import pandas as pd

pdf = pd.read_csv("manufacturing_production_120k.csv")
```

Exercises:

1. Display first 5 rows.
2. Display last 5 rows.
3. Display shape.
4. Display column names.
5. Display datatypes.
6. Select `plant_id`, `machine_id`, `units_produced`.
7. Rename `units_produced` to `production_output`.
8. Filter rows where `quality_status == "Reject"`.
9. Filter rows where temperature is greater than 75.
10. Filter Night-shift production from Hyderabad plant.

---

4. Dask DataFrame

Use:

```
import dask.dataframe as dd

ddf = dd.read_csv(
    "manufacturing_production_120k.csv"
)
```

Dask DataFrame partitions tabular data into multiple pandas DataFrames and normally evaluates lazily, so operations often require `.compute()` to produce the result. docs.dask.org

Exercises:

1. Display first records using `head()`.
2. Find number of partitions.

3. Select `plant_id`, `product_family`, `units_produced`.
4. Filter rejected production records.
5. Calculate average units produced by plant.
6. Calculate total defects by product family.
7. Use `.compute()` to obtain the final result.
8. Compare doing the same aggregation in Pandas and Dask.

For data comfortably fitting in RAM, Pandas can often remain simpler; Dask becomes useful when parallel or larger-than-memory processing matters. [docs.dask.org]

---

5. Manipulating DataFrames

Using PySpark:

1. Create:

```
production_efficiency =
units_produced / units_planned * 100
```

2. Create:

```
good_units =
units_produced - defective_units
```

3. Add:

```
country = India
```

4. Rename `energy_kwh` to `energy_consumed`.
5. Convert `units_produced` to double.
6. Create `defect_percentage`.
7. Create `energy_per_unit`.
8. Select only the transformed columns.

---

6. Dropping Data

1. Drop `supervisor`.
2. Drop `supervisor` and `maintenance_flag`.
3. Remove duplicate rows.
4. Keep one record per `production_id`.
5. Remove records where operator ID is missing.
6. Remove rows where both temperature and vibration are missing.
7. Display unique product families.
8. Display unique plant-machine combinations.

---

7. Sorting

1. Sort by units produced ascending.
2. Sort units produced descending.
3. Sort highest defective units first.
4. Sort by plant and then production output descending.
5. Display the top 20 production runs.
6. Display the 10 runs with highest downtime.
7. Find machines with the highest recorded vibration.

---

8. Aggregations

Calculate:

1. Total units planned.
2. Total units produced.
3. Total defective units.
4. Average units produced.
5. Maximum production output.
6. Minimum production output.
7. Average downtime.
8. Total energy consumed.
9. Average temperature.
10. Average vibration.

---

9. `groupBy()`

Find:

1. Total production by plant.
2. Total production by product family.
3. Total defects by plant.
4. Average downtime by shift.
5. Average energy usage by product family.
6. Production count by quality status.
7. Production by plant and shift.
8. Defects by plant and product family.
9. Average temperature by machine.
10. Total output by supervisor.

Then sort each summary appropriately.

---

10. Joining DataFrames

Load:

```
plant_df = spark.read.csv(
    "plant_master.csv",
    header=True,
    inferSchema=True
)

machine_df = spark.read.csv(
    "machine_master.csv",
    header=True,
    inferSchema=True
)
```

Exercises:

1. Inner join production with plant master.
2. Display plant name instead of only plant ID.
3. Join production with machine master.
4. Create one DataFrame containing production + plant + machine information.
5. Calculate total production by plant city.
6. Calculate average downtime by machine type.
7. Calculate defects by region.
8. Find production records without matching machine master records.

9. Perform a left join and inspect unmatched records.
10. Find total energy consumption by plant name.

---

11. Updating Data

Remember that Spark transformations create new DataFrames rather than modifying rows in place.

Exercises:

1. Change `Reject` to `Failed`.
2. Change `Rework` to `Requires Review`.
3. Increase units planned by 5% for Motors.
4. Increase energy consumption by 3% for Night shifts.
5. Replace missing operator IDs with `UNASSIGNED`.
6. Replace supervisor `Anita` with `Anita Rao`.
7. Create a corrected temperature column.

---

12. Null Values and `na` Functions

The dataset deliberately contains null/blank values.

Exercises:

1. Count missing operator IDs.
2. Count missing temperature values.
3. Count missing vibration values.
4. Count missing downtime.
5. Use `na.drop()`.
6. Drop only rows where temperature is missing.
7. Fill missing downtime with `0`.
8. Fill missing operator ID with `UNKNOWN`.
9. Fill missing temperature with the overall average temperature.
10. Compare record counts before and after cleansing.

Also practice:

```
na.drop()
na.fill()
```

```
fillna()
isNull()
isNotNull()
coalesce()
```

---

13. Built-in Functions

Practice these PySpark functions:

```
upper
lower
trim
length
concat
concat_ws
round
abs
coalesce
regexp_replace
substring
```

Exercises:

1. Convert product family to uppercase.
2. Convert supervisor names to lowercase.
3. Trim operator IDs.
4. Find length of product codes.
5. Combine plant ID and machine ID.
6. Round temperature to one decimal.
7. Calculate absolute difference between planned and produced units.
8. Replace characters in production ID using `regexp_replace`.
9. Extract part of product code using `substring`.
10. Use `coalesce()` to return temperature or a default value.

---

14. `when()` / `otherwise()`

Create:

Efficiency Category

```
>= 100%  → Above Target
>= 95%   → On Target
>= 85%   → Below Target
otherwise → Critical
```

Defect Category

```
0               → Perfect
1-10            → Normal
11-25           → Attention
above 25        → High Defect
```

Machine Condition

Use temperature and vibration:

```
High temperature OR high vibration → Inspect
otherwise                          → Normal
```

Then count records in each resulting category.

---

15. Complex JSON

Use:

:chatgpt-content-reference{index="9"}Download machine_configurations.json

Read:

```
machine_json = spark.read.option(
    "multiline", "true"
).json("machine_configurations.json")
```

Exercises:

1. Display schema.
2. Display machine ID and machine type.
3. Extract `specifications.rated_power_kw`.
4. Extract maximum temperature.
5. Extract vibration limit.
6. Display maintenance frequency.
7. Work with the `sensors` array.
8. Explode the sensors array.
9. Display one row per machine/sensor combination.
10. Find machines having an active temperature sensor.
11. Join machine configuration JSON with production data.

---

16. Union and Union ALL

Load:

```
extra_df = spark.read.csv(
    "manufacturing_production_extra_12k.csv",
    header=True,
    inferSchema=True
)
```

Exercises:

1. Count records in the main DataFrame.
2. Count records in the extra DataFrame.
3. Combine using `union()`.
4. Combine using `unionAll()`.
5. Compare resulting counts.
6. Identify duplicate `production_id` values.
7. Remove duplicates after combining.
8. Compare unioned count before and after `dropDuplicates()`.
9. Try `unionByName()`.
10. Rearrange columns in one DataFrame and use `unionByName()` appropriately.

---

17. Window Functions

Use:

```
from pyspark.sql.window import Window
```

Exercises:

1. Rank production records within each plant by units produced.
2. Use `row_number()` .
3. Use `rank()` .
4. Use `dense_rank()` .
5. Find top 5 production runs per plant.
6. Find top 3 machines by output within each plant.
7. Calculate total plant production without collapsing rows.
8. Calculate average machine output using a window.
9. Calculate cumulative production by machine ordered by date.
10. Display previous production output using `lag()` .
11. Display next production output using `lead()` .
12. Calculate change from the previous production run.
13. Rank product families by total output.

---

18. Date and Time Functions

Using `production_date` :

1. Convert to `DateType` .
2. Extract year.
3. Extract month.
4. Extract day.
5. Extract month name.
6. Extract day name.
7. Create `year_month` .
8. Calculate monthly production.
9. Calculate monthly defects.
10. Calculate production by month and plant.
11. Find production between two dates.
12. Add 7 days to production date.
13. Subtract 30 days.
14. Calculate difference from current date.
15. Find the month with the highest total output.